

Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial

Instituto Tecnológico de Buenos Aires



Grupo EZduino (G-129)

Integrantes:

- Arias, Mateo (65613)
- Garcia Puente, Manuel (65505)
- González Núñez, Pedro (64467)
- Spivak Fontaiña, Federico (60538)

Fecha de Entrega: 14/06/2026

	2
1. Introducción	3
2. Modelo Computacional	3
2.1 Dominio	3
2.2 Lenguaje	3
3. Implementación	4
3.1 Frontend	4
3.2 Backend	5
3.3 Dificultades Encontradas	6
4. Futuras Extensiones	7
5. Conclusiones	7
6. Referencias	8
7. Bibliografía	8

1. Introducción

El presente trabajo consiste en el desarrollo de un compilador con el objetivo de permitir la programación de placas Arduino mediante el lenguaje EZduino en el marco de la materia Autómatas, Teoría de Lenguajes y Compiladores. El objetivo principalmente fue brindarle al usuario una opción más sencilla para describir el comportamiento de componentes electrónicos como LEDs, sensores, actuadores y pantallas, abstrayendo los detalles de configuración de pines e inicialización de periféricos.

El compilador fue desarrollado en el lenguaje de programación C. Una de las principales herramientas empleadas fue Flex para la definición y generación del analizador léxico, el cual se encarga de identificar los distintos componentes léxicos del lenguaje de entrada. Otra importante herramienta utilizada fue Bison, que se empleó para construir el analizador sintáctico el cual valida que la estructura del programa EZduino siga la gramática definida, además de construir las estructuras internas necesarias para la posterior generación del código C++ para Arduino.

El proceso se separó en tres etapas. La primera consistió en la propuesta y diseño de la idea principal, donde se pensaron requerimientos, propuesta, ejemplos y casos de prueba. En la segunda etapa, se llevó adelante el desarrollo del frontend, que representa "el frente" del compilador, es decir el analizador léxico y sintáctico. El principal objetivo de esta etapa fue la construcción del AST (Árbol de Sintaxis Abstracta) de la entrada. Finalmente, la tercera etapa constó del acabado de algunos aspectos del frontend y validaciones semánticas, para luego generar el código deseado por el usuario.

2. Modelo Computacional

2.1 Dominio

El dominio del problema se centra en la configuración de hardware y la lógica de control reactiva en sistemas embebidos, específicamente en la plataforma Arduino. Los elementos del dominio incluyen componentes físicos como sensores (ultrasónicos para medir distancia, DHT11 para temperatura y humedad), actuadores (LEDs, buzzers, servomotores), potenciómetros y pantallas LCD, cada uno con sus propios pines de conexión —ya sean analógicos o digitales—, protocolos de comunicación y métodos específicos de lectura o escritura. A esto se suma el ciclo de vida del microcontrolador, compuesto por una etapa de inicialización donde se configuran los modos de los pines y se inicializan los periféricos, y un bucle de ejecución continuo donde se efectúa la lógica de control del programa, incluyendo la lectura de sensores, el accionamiento de actuadores y la gestión de tiempos mediante retardos o ejecución periódica.

2.2 Lenguaje

El lenguaje EZduino se diseñó con una sintaxis declarativa y estructurada en dos bloques principales: el bloque *hardware* y el bloque *routine*. En el primero se declaran los componentes físicos conectados a la placa Arduino, especificando su tipo (led, buzzer, button, potentiometer, servo, ultrasonic, dht11, lcd), un nombre identificador y los pines de conexión correspondientes. Cada tipo de componente tiene su propia forma de especificar

pinos: los componentes simples reciben un único número de pin, mientras que otros como el *ultrasonic* requieren pines nombrados (*trig* y *echo*) y el *lcd* requiere una lista de seis pines con nombre.

El bloque *routine* contiene la lógica de control del programa, donde se pueden declarar variables con tipado inferido (*var x = 10*), realizar asignaciones, invocar métodos sobre los componentes declarados (por ejemplo, *sensor.read_temperature()*) y utilizar estructuras de control como, *repeat every*, *repeat times*, *for-from-to* y *wait*. Las expresiones soportan operadores aritméticos (+, -, *, /), relacionales (==, !=, <, >, <=, >=), lógicos (and, or, not) y literales de tiempo (como *2s* o *100ms*), permitiendo construir programas completos para control de hardware embebido.

Como lenguaje de entrada, EZduino busca ser conciso y legible, abstrayendo los detalles de configuración de pines e inicialización de periféricos. Como lenguaje de salida, el compilador genera código C++ orientado a Arduino (archivos *.ino*), que incluye las directivas *#define* para los pines, las declaraciones de las librerías y objetos necesarios para cada componente, la función *setup()* con las inicializaciones correspondientes y la función *loop()* con la lógica de control traducida, incluyendo la gestión de tiempos mediante *millis()* para las ejecuciones periódicas.

3. Implementación

Para la implementación del compilador se utilizó como base el proyecto [Flex-Bison-Compiler](#) (sobre la rama *development*), que facilitó el uso de las herramientas Flex (para el análisis léxico) y Bison (para el análisis sintáctico), como ya se describió previamente.

3.1 Frontend

El frontend del compilador se encarga del análisis del código fuente EZduino y está compuesto por dos módulos principales: el analizador léxico y el analizador sintáctico. El primero fue implementado con Flex y se encarga de recorrer el archivo de entrada identificando los distintos componentes léxicos del lenguaje, tales como palabras clave (*hardware*, *routine*, *if*, *repeat*, *var*, etc.), tipos de componentes (*led*, *buzzer*, *ultrasonic*, *lcd*, etc.), operadores, literales (enteros, flotantes, strings, booleanos y literales de tiempo), identificadores y símbolos. Cada lexema reconocido es convertido en un token con su etiqueta, valor semántico y posición en el archivo, y luego es enviado al analizador sintáctico.

El analizador sintáctico fue implementado con Bison y valida que la secuencia de tokens recibida cumpla con la gramática definida para el lenguaje EZduino. A medida que reconoce las producciones de la gramática, ejecuta acciones semánticas que construyen el Árbol de Sintaxis Abstracta (AST), una representación en memoria del programa que incluye el bloque de hardware con sus declaraciones de componentes y pines, y el bloque de rutina con sus sentencias y expresiones. Este AST es la estructura de datos que luego utiliza el backend para realizar el análisis semántico y la generación de código.

3.2 Backend

El backend del compilador está compuesto por dos módulos: el analizador semántico y el generador de código. El analizador semántico trabaja en dos pasadas sobre el AST construido por el frontend. En la primera pasada, recorre el árbol recolectando todas las declaraciones de componentes de hardware y variables, y las registra en una tabla de símbolos con soporte de ámbitos anidados. En esta etapa se validan aspectos como la declaración duplicada de componentes, la cantidad y tipo de pines esperados para cada componente (por ejemplo, que un *ultrasonic* tenga exactamente dos pines nombrados *trig* y *echo*, o que un *lcd* tenga seis pines con los nombres *rs*, *en*, *d4*, *d5*, *d6*, *d7*), el rango válido de pines digitales y analógicos, y que no haya pines duplicados entre distintos componentes. En la segunda pasada, se realiza la verificación de tipos de todas las expresiones y sentencias, incluyendo la validación de que los métodos invocados sobre cada componente existan y reciban los argumentos correctos en cantidad y tipo.

El generador de código recorre el AST ya validado y produce un archivo *.ino* en C++ para Arduino. Para ello, genera las directivas *#include* de las librerías necesarias (*LiquidCrystal*, *DHT*, *Servo*), las macros *#define* para los pines de cada componente, las declaraciones globales de los objetos (como *DHT*, *Servo*, *LiquidCrystal*), la función *setup()* con las inicializaciones correspondientes (*pinMode*, *attach*, *begin*), y la función *loop()* con la lógica de control traducida, incluyendo la gestión de tiempos mediante *millis()* para las ejecuciones periódicas y la función auxiliar *readUltrasonicDistance* para el sensor ultrasónico.

Bloque de código 1: Ejemplo de scaffolding generado y puntos de extensión (hooks).

```
void setup() {
  // ... código autogenerado ...
  // === Add custom setup code here ===
}

void loop() {
  // ... código autogenerado ...
  // === Add custom loop code here ===
}
```

- Mapeo de pines multipin a C++:

Para evitar el "hardcodeo" de pines en el código final, el generador utiliza una convención estricta para nombrar las macros *#define*: el prefijo es *PIN_* seguido del nombre del componente en mayúsculas. Para componentes multipin, se agrega el nombre original del pin como sufijo.

Tabla 1: Convención de generación de macros de pines

EZduino	#define generado
led status on 13	#define PIN_STATUS 13
ultrasonic radar on (trig: 9, echo: 10)	#define PIN_RADAR_TRIG 9 #define PIN_RADAR_ECHO 10
lcd screen on (rs: 12, en: 11, d4: 5, d5: 6, d6: 3, d7: 2)	#define PIN_SCREEN_rs 12 ... #define PIN_SCREEN_d7 2
servo barrier on 6	#define PIN_BARRIER 6

El programador puede luego hacer uso de estas macros (ej. PIN_RADAR_TRIG) si necesita extender la lógica C++ generada por EZduino.

- Generación de inicialización e instancias:

Tabla 2: Traducción de componentes a C++

Componente EZduino	Global generado	setup() generado
led led1 on P	#define PIN_LED1 P	pinMode(PIN_LED1, OUTPUT)
button b1 on P	#define PIN_B1 P	pinMode(PIN_B1, INPUT)
servo s1 on P	#include <Servo.h> #define PIN_S1 P Servo s1	s1.attach(PIN_S1)
lcd l1 on (rs:R, ...)	#include <LiquidCrystal.h> #define PIN_L1_rs R ... LiquidCrystal l1(...)	l1.begin(16, 2)
buzzer bz1 on P	#define PIN_BZ1 P	pinMode(PIN_BZ1, OUTPUT)
potentiometer p1 on P	#define PIN_P1 P	—
dht11 d1 on P	#include <DHT.h> + #define PIN_D1 P + DHT d1(PIN_D1, DHT11)	d1.begin()
ultrasonic u1 on (trig: T, echo: E)	#define PIN_U1_TRIG T + #define PIN_U1_ECHO E + función helper	pinMode(PIN_U1_TRIG, OUTPUT) + pinMode(PIN_U1_ECHO, INPUT)

3.3 Dificultades Encontradas

Una dificultad surgió con las variables de la routine que se inicializan con llamadas a métodos de hardware, por ejemplo: `var temp = sensor.read_temperature()`. Estas no pueden traducirse como inicializadores globales en C++ porque los objetos de hardware (como el bus de sensores o pines) no están listos hasta que se ejecuta el `setup()`.

Para solucionarlo, el compilador divide la declaración: se emite una variable global con un valor default seguro (como 0.0, 0 o false), y la asignación con la lectura en tiempo de ejecución se genera dentro de la función `loop()`.

4. Futuras Extensiones

- **Implementación de wait no bloqueante:** Actualmente, la instrucción wait se traduce a la función delay() de Arduino, lo cual bloquea el flujo de ejecución del microcontrolador. Para adherir a la especificación ideal no bloqueante, se planea implementar un sistema basado en máquinas de estado implícitas, donde cada llamada a loop() evalúa temporizadores en lugar de detener el procesador.
- **Soporte para pantallas LCD I2C:** Por el momento el LCD de EZduino funciona mediante configuración en paralelo (requiriendo 6 pines). Podría extenderse el lenguaje y el backend para incluir soporte para módulos I2C integrando la librería LiquidCrystal_I2C, simplificando enormemente el conexionado (solo SDA y SCL).

5. Conclusiones

El desarrollo del compilador para el lenguaje EZduino cumplió exitosamente con los objetivos planteados en el marco de la materia, logrando materializar una herramienta capaz de abstraer la complejidad asociada a la programación de sistemas embebidos en Arduino. A través de una arquitectura bien definida, la combinación de Flex y Bison en el frontend permitió estructurar un analizador léxico y sintáctico sólido para la construcción del Árbol de Sintaxis Abstracta (AST). Por su parte, el diseño del backend y su analizador semántico en dos pasadas resultaron fundamentales para garantizar la robustez del sistema, resolviendo con éxito desafíos críticos como la validación de pines y la correcta asignación temporal de variables globales dentro del ciclo de ejecución del microcontrolador.

Más allá de los logros técnicos en la traducción de la lógica declarativa a código C++ funcional, el proyecto brindó una experiencia invaluable en el diseño de software de base y en la gestión del ciclo de vida del hardware. La resolución de limitaciones físicas y la integración de librerías nativas demostraron la viabilidad de EZduino, dejando al mismo tiempo las bases asentadas para futuras expansiones de alto valor.

El proyecto permitió una visualización más concreta de las temáticas estudiadas en la materia, permitiendo elegir el dominio y los lenguajes, y desarrollar los análisis sintácticos, léxicos y semánticos, así también como realizar la generación de código. El proyecto base provisto por la cátedra fue una herramienta fundamental para el cumplimiento de la tarea asignada, aunque representó un nuevo desafío ya que se basaba en tecnologías con las que nunca se había trabajado previamente (Flex y Bison).

6. Referencias

- https://github.com/pedrogonzaleznunez/TPE_TLA/tree/development

7. Bibliografía

-  (2026-03-12) Proyecto Especial
-  (2026-03-12) Modelo Computacional
-  (2025-09-03, v1.0.0) Análisis Léxico
-  (2024-05-08, v0.1.0) Análisis Sintáctico
-  (2024-09-25, v0.2.0) Análisis Semántico
-  (2024-05-16, v0.1.0) Generación de Código
-  (2024-05-16, v0.1.0) Runtime
-  (2024-05-16, v0.1.0) Tabla de Símbolos
-  (2024-05-16, v0.1.0) Scopes
-  (2024-05-16, v0.1.0) Sistema de Tipos
- Documentación de referencia de Arduino: <https://www.arduino.cc/reference/en/>